

פרויקט במבוא לבינה מלאכותית - אורי דוד סטי ויונתן ברק

1. קישור למסד הנתונים בו השתמשתם, וכן תיאור של הנתונים (סוג הנתונים, כמות הדגימות, באלו פיצ'רים השתמשתם מתוך הנתונים וכו').

- קישור למסד הנתונים בו השתמשנו -

<https://www.kaggle.com/datasets/mysarahmadbhat/airline-passenger-satisfaction>

מערך הנתונים ששימש אותנו הוא סקר שביעות רצון שנערך בקרב נוסעי חברת תעופה. מטרת הפרויקט היא לבנות מודל סיווג (Classification) שינבא את שביעות הרצון של הלקוח ('satisfied' או 'Neutral or Dissatisfied') על סמך מאפייניו ומאפייני הטיסה.

כמות הדגימות: מערך הנתונים המלא כולל כ-130,000 דגימות (שורות), כאשר כל דגימה מייצגת נוסע בודד. לצורך בניית המודלים, הנתונים חולקו לקבוצות אימון, ולידציה ומבחן. קבוצת האימון, לאחר טיפול בנתונים חסרים וקידוד, מכילה 90,916 דגימות.

הפיצ'רים (Features) בהם נעשה שימוש: לאחר ניקוי ראשוני של עמודות לא רלוונטיות (כמו id), השתמשנו ב-23 פיצ'רים כדי לנבא את שביעות הרצון.

שם השדה בעברית	תיאור	שם השדה באנגלית
מזהה	מזהה ייחודי לכל נוסע	ID
מגדר	המגדר של הנוסע (זכר/נקבה)	Gender
גיל	גיל הנוסע	Age
סוג נוסע	סוג נוסע (חוזר או פעם ראשונה)	Customer Type
מטרת הנסיעה	מטרת הנסיעה של הנוסע (עסקית או אישית)	Type of Travel
מחלקה	המחלקה שבה המושב	Class

	של הנוסע	
Flight Distance	מרחק הטיסה ביחידות mile	מרחק טיסה
Departure Delay	זמן עיכוב בהמראה בדקות	עיכוב בהמראה
Arrival Delay	זמן עיכוב הנחיתה בדקות	עיכוב בנחיתה
Departure and Arrival Time Convenience	רמת שביעות רצון מנוחות זמני ההמראה והנחיתה מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	נוחות זמן המראה ונחיתה
Ease of Online Booking	רמת שביעות רצון מקלות ההזמנה באונליין מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	קלות ההזמנה באונליין
Check-in Service	רמת שביעות רצון משירות הצ'ק אין מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	שירות הצ'ק אין
Online Boarding	רמת שביעות רצון משירות הצ'ק אין אונליין מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	שירות צ'ק אין אונליין
Gate Location	רמת שביעות רצון ממיקום הגייט מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	מיקום הגייט
On-board Service	רמת שביעות רצון מהשירות במטוס מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	השירות במטוס
Seat Comfort	רמת שביעות רצון מנוחות הכיסא במטוס מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	נוחות הכיסא

Leg Room Service	רמת שביעות רצון מהמקום לרגליים במושב מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	מקום לרגליים
Cleanliness	רמת שביעות רצון מנקיון המטוס מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	נקיון
Food and Drink	רמת שביעות רצון מהאוכל והשתייה בטיסה מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	אוכל ושתייה
In-flight Service	רמת שביעות רצון מרמת השירות במטוס מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	שירות במטוס
In-flight Wifi Service	רמת שביעות רצון משירות הwifi בטיסה מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	שירות הWifi במטוס
In-Flight Entertainment	רמת שביעות רצון מהבידור בטיסה מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	בידור במהלך הטיסה
Baggage Handling	רמת שביעות רצון מהטיפול בכבודה מ1 עד 5 ו0 משמעותו שלא התקבל מידע.	טיפול בכבודה
Satisfaction	רמת שביעות רצון כללית מהחברה (מרוצה / ניטרלי או לא מרוצה)	שביעות רצון

2. תיאור האלגוריתמים ושיטות הלמידה בהן השתמשתם, וכן הסבר מדוע בחרתם בשיטות אלה ומדוע הן מתאימות לסוג בנתונים וסוג הניתוח הרצוי

במהלך העבודה על הפרויקט השתמשנו במספר שיטות למידה במטרה לבנות מודל שינבא באופן טוב את שביעות הרצון של נוסעים. התחלנו עם מודל פשוט של רגרסיה לוגיסטית, שהוא קל ומהיר לאימון וגם ישמש כמודל להשוות אליו את הבאים. לאחר מכן השתמשנו באלגוריתם יער אקראי כדי לבדוק מודל כבד יותר ואת יכולת הניבוי שלו על הנתונים. ולבסוף יצרנו מודל של רשתות נוירונים שהוא מודל כבד ומרכזי בפרויקט כפי שהומלץ להשתמש בו בהנחיות המטלה.

רגרסיה לוגיסטית:

רגרסיה לוגיסטית היא אלגוריתם סיווג שמשמש לניבוי תוצאה של משתנה בינארי בהינתן המשתנים הבלתי תלויים. כלומר אנו רוצים למצוא תלות של הסתברות של משתנה בינארי Y בקבוצת משתנים בינאריים $X_1, \dots, X_n$ תוך הנחה שהם בלתי תלויים בהינתן Y . במקרה שלנו אנו מרוצה או לא מרוצה הוא המשתנה הבינארי וקיימים 22 משתנים בלתי תלויים המשפיעים. תחילה המודל פועל בדומה לרגרסיה רב לינארית, הוא מוצא כך מקדמים a ו- b המקיימים את מה שמופיע בתמונה.

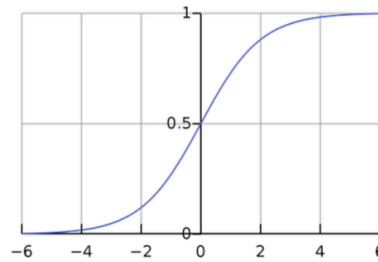
$$\log \frac{P(Y = 1|X_1, \dots, X_n)}{P(Y = 0|X_1, \dots, X_n)} = \sum_{i=1}^n a_i X_i + b$$

לאחר מכן הוא משתמש בתוצאה של המקדים המקסימלים לפי הרגרסיה הרב לינארית את ההסתברות שמאורע $Y = 1$ מתקיים לפי המשתנים הבלתי תלויים. כלומר מה ההסתברות שהוא מרוצה. ואז מקבלים את המשוואה הבאה בתמונה:

$$P(Y = 1|X_1, \dots, X_n) = \frac{1}{1 + \exp(-(\sum_{i=1}^n a_i X_i + b))}$$

בשוויון מצד ימין מופיעה הפונקציה הלוגיסטית שמופעלת על הערך שהרגרסיה הרב לינארית נתנה לנו. כך היא הופכת את הערכים לבין 0 ל-1 כלומר להסתברות שהמאורע נכון. כלומר הפונקציה לוקחת מספרים חיוביים ושליליים ומחזירה מספר בין 0 ל-1. כאשר כמה שהוא יותר גדול הוא

מתקרב ל1. ניתן לראות את זה גם לפי גרף הפונקציה:



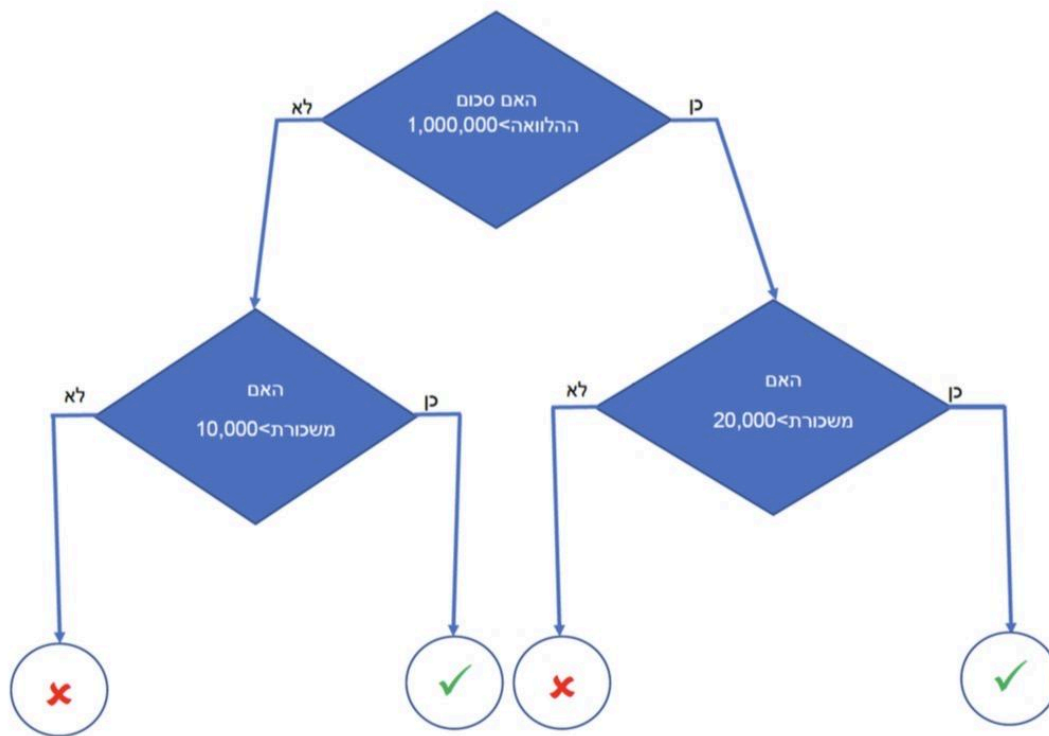
לבסוף לבחירה של הסיווג קובעים הסתברות למאורע $Y=1$ שהחל מהסתברות כזאת או גדולה מזאת קובעים $Y=1$ ואחרת $Y=0$.

בחרנו לעשות את האלגוריתם של רגרסיה לוגיסטית כמודל בסיסי אשר יביא לנו מידע על הקשר הבסיסי בין המשתנים ויהווה לנו כייחוס עבור מודלים מורכבים יותר. בנוסף ההנחה של לינאריות ברגרסיה לינארית יכולה להיות מגבילה עבור הנתונים שלנו אבל היא תוכל להראות לנו את הקשר בין הנתונים ולהוות לנו בסיס השוואה באמצעות מודל מהיר ופשוט.

יער אקראי:

יער אקראי הוא אלגוריתם למידת פופולרי, שמסתמך על מספר אלגוריתמי סיווג חלשים יותר. במקום להסתמך על מודל בודד, הוא בונה מספר רב של עצי החלטה ומשלב את התחזיות של כולם כדי להגיע להחלטה סופית ומדויקת יותר.

עץ החלטה בודד הוא מודל סיווג פשוט. הוא מקבל החלטות החלטות על ידי סדרת שאלות של כן ולא על הפיצ'רים השונים עד לקבלת החלטה. כפי שניתן לראות באיור הבאה:



עם זאת, לעצי החלטה בודדים יש חסרונות משמעותיים: זה אלגוריתמים greedy, הפ רגישים מאוד לשינויים קטנים בנתוני האימון, ונוטים overfitting.

ניכר האקראי מנסה לפתור את בעיות אלה. הוא מתגבר על חסרונות אלו על ידי בניית יער שלם של עצים, כאשר כל עץ נבנה באופן מעט שונה כדי להבטיח גיוון:

1. כל עץ מאומן על תת מדגם אקראי של נתוני האימון (תהליך הנקרא Bootstrap).
2. בכל פיצול בעץ, נלקחת בחשבון רק תת קבוצה אקראית של הפיצורים.

תהליך זה מבטיח שהעצים ביער יהיו שונים זה מזה. הניבוי הסופי של המודל מתקבל על ידי מה שרוב העצים מסווגים. כל עץ נותן את התחזית שלו, והתחזית שקיבלה את מירב הקולות היא התוצאה הסופית. גישה זו מפחיתה באופן משמעותי את הרעש והשונות, ומובילה למודל יציב, חזק, ובעל יכולת הכללה טובה יותר. האלגוריתם כולל 3 פרמטרים עיקריים שיש להגדיר לפני הביצוע: מספר העצים, גודל הצומת ומספר התכונות שנדגמות.

סיבת הבחירה והתאמה לנתונים: המעבר למודל יער אקראי נעשה מתוך הנחה ההגיונית שהקשר בין שביעות רצון הנוסעים לבין המאפיינים השונים אינו בהכרח לינארי. בנוסף היה ניתן לראות בעת ביצוע הרגרסיה הלוגיסטית שהתוצאות לא מושלמות. בניגוד לרגרסיה לוגיסטית, יער אקראי מסוגל ללמוד קשרים מורכבים ואינטראקציות בין פיצורים שונים. לדוגמה, ייתכן ששירותי wifi מגורע מפריע לנוסעי מחלקת עסקים הרבה יותר מאשר לנוסעים במחלקת תיירים. יער אקראי יכול ללמוד תבניות כאלה באופן אוטומטי. יכולתו להתמודד עם נתונים מגוונים והעמידות הגבוהה שלו בפני

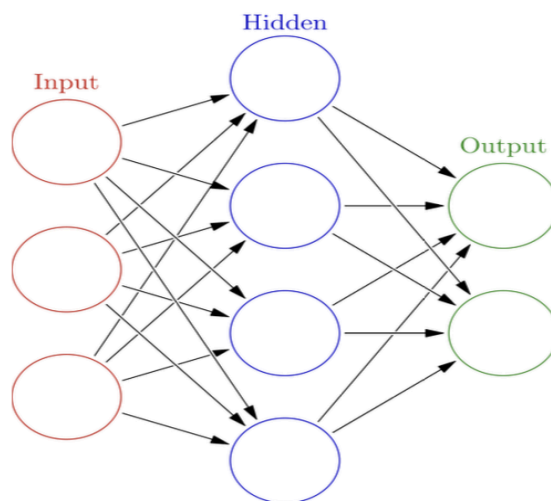
overfitting הופכות אותו לבחירה הגיונית כמודל מתקדם, המסוגל לספק שיפור משמעותי בביצועים על פני מודל הבסיס הלינארי.

רשתות נוירונים:

רשתות נוירונים זה מודל מתקדם אשר מקבל השראה ממודל פשטני של פעילות הנוירונים במוח. המודל מורכב מיחידות אשר מדמות את הנוירונים והן מאורגנות בשכבות מקושרות. כל נוירון מקבל מספר קלטות ומחשב סכום שלהם עם מקדמים ולפי זה מחליט אם הוא דולק או מכובה. כלומר כמו בפונקציה שבתמונה:

$$f(x_1, \dots, x_k) = \theta \left(\sum w_i x_i - b \right) := \begin{cases} 1 & \sum w_i x_i > b \\ 0 & \sum w_i x_i \leq b \end{cases}$$

כפי שצינו הנוירונים מחולקים למספר שכבות כאשר השכבה הראשונה הראשונה היא שכבת הקלט המקבלת את הקלט ומספרה הוא כמספר הקלטות כלומר אצלנו 23 נוירונים בשכבה הראשונה כמספר הקלטות. הסוג השני של השכבות הוא hidden layers שהן השכבות הביניים יכול להיות מספר שכבות מסוג זה במודל. בשכבות אלה מתבצע עיקר התהליך של הלמידה וכל שכבה כזאת לומדת לזהות דפוס מורכב יותר של התנהגות. הסוג השלישי הוא שכבת הפלט, בה נמצא הסיכוי של כל פלט לצאת, כיוון שאצלנו זה פלט בינארי השכבה הזאת משתמשת בנוירון בודד. ניתן לראות גם את מבנה השכבות בשרטוט הבא:



תהליך הלמידה מתבצע על ידי כך שהרשת משווה את הניבוי שלה לתוצאה האמיתית ומחשבת את פונקציית השגיאה. לאחר מכן היא משתמשת בשיטת האופטימיזציה gradient descent כדי למזער

את השגיאה על ידי שינוי המשקלים אשר משפיעם על הפעלת כל נוירון עד להגעת משקלים אופטימליים.

בחרנו לבנות רשת נוירונים כמודל ה"כבד" והמתקדם ביותר בפרויקט, בהתאם להמלצות בהנחיות הקורס. רשתות נוירונים הן כלי רב עוצמה המסוגל לגלות את הקשרים מורכבים ביותר בנתונים, כאלה שגם מודל כמו יער אקראי יכול לפספס. כמו כן, בניית רשת נוירונים אפשרה לנו להדגים באופן מעשי את ההתמודדות עם האתגר המרכזי של מודלים מורכבים over-fitting וליישם טכניקות מתקדמות למניעתה כמו: Early Stopping וDropout, כפי שנדרש בסעיף 6 של הנחיות הפרויקט.

3. תיאור של פונקציית הסיכון בה השתמשתם, הדרך בה הערכתם את הביצועים, וכיצד בחרתם הפרמטרים של האלגוריתם בהתאמה למטרות.

בפרויקט זה, פונקציית הסיכון מתייחסת לפונקציות ההפסד שהמודלים ממזערים.

עבור הרשת העצבית השתמשנו במפורש ב־פונקציית הפסד של אנטרופיה צולבת בינארית (Binary Cross-Entropy), המוגדרת כך:

$$L(y, \hat{y}) = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})],$$

כאשר $y \in \{0, 1\}$ הסיווג האמיתי ו־ $\hat{y}$ הסיווג המחושב.

רגרסיה לוגיסטית ממזערת את אותה פונקציה, אך בממוצע על פני כל הדגימות:

$$R(f) = \frac{1}{n} \sum_{i=1}^n -[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)],$$

משוואה זו מייצגת את הסיכון האמפירי, כלומר השגיאה הממוצעת של המודל על הדאטה.

יער אקראי (Random Forest) אינו ממזער אנטרופיה צולבת ישירות, אלא בוחר פיצולים שממקסמים את טוהר הצמתים באמצעות אינדקס ג'יני:

$$Gini = 1 - \sum_{k=1}^K p_k^2,$$

כאשר p_k הוא השיעור של כל מחלקה בצומת.

בסופו של דבר, ערך פונקציית הסיכון מבטא את רמת הטעות הממוצעת של המודל: ערך נמוך מצביע על התאמה טובה יותר של החיזויים לנתונים, בעוד ערך גבוה מצביע על התאמה פחות טובה.

הערכנו את הביצועים לפי ROC-AUC כאשר הרגשנו שנתון זה ייצג תמונה רחבה וטובה להסכלות על המודל. בהתאם, בעת חיפוש הפרמטרים הטובים ביותר הרצנו חיפושים (נפרט יותר בחלק 7) עם שיטת ניקוד של ROC-AUC כך שנקבל לבסוף את הפרמטרים הגורמים לערך זה להיות גבוה ביותר.

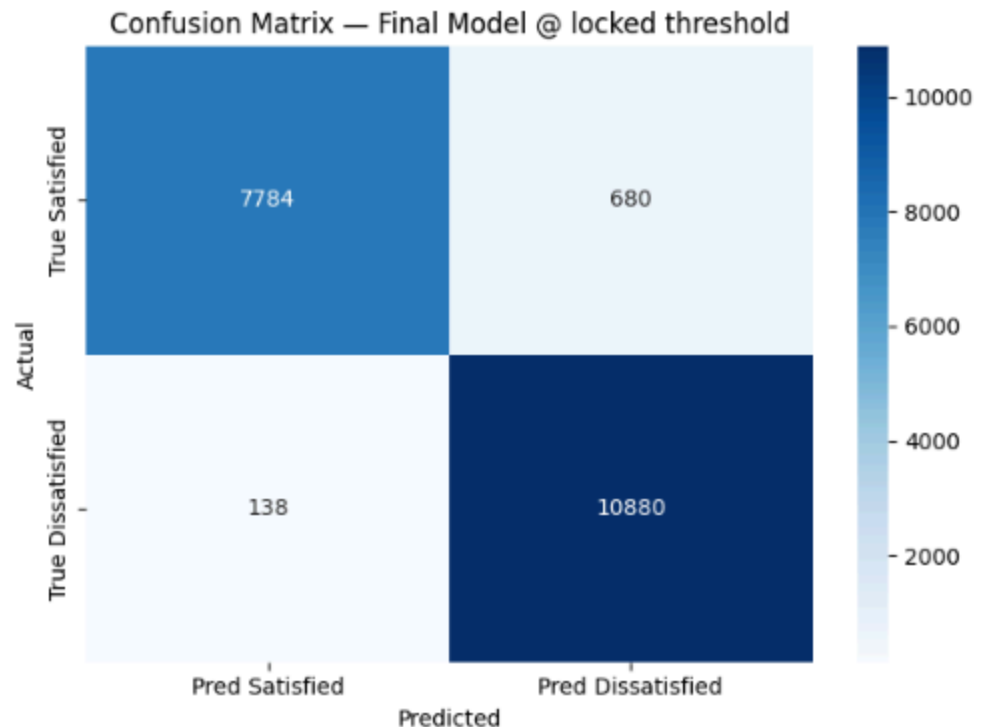
בחרנו את הפרמטרים ההתחלתיים עבור כל אלגוריתם באופן סטנדרטי, כך שיהווה לנו מקור השוואה טוב בין האלגוריתמים וייתן לנו נקודת פתיחה הגיונית שממנה נשפר את האלגוריתם.

עבור רגרסיה לינארית ויער אקראי השתמשנו בפרמטרים הבסיסיים המוגדרים כברירת מחדל בספרייה. בחרנו זאת כבחירה סטנדרטית וכדי לבנות את מודל הבסיס במהירות, שיביא לנו מודל בסיסי.

עבור רשת נוירונים פעלנו בצורה הבאה: מבנה השכבות באמצע בחרנו ל64 ואז 32 נוירונים כאשר הראשונה 64 והשנייה 32. כיוון שמבנה זה מאפשר בהתחלה ללמוד תבניות רחבות ומגוונות ואז לזקק אותן למספר הקטן בחצי. בשכבות החבויות בחרנו להשתמש בפונקציות ReLU שמאפשרת למודל ללמוד דפוסים לא לינאריים וזו הפונקציה הסטנדרטית והמקובלת. ולשכבת הפלט בחרנו בפונקציית sigmoid שמביאה תוצאה בינארית. בנוסף בחרנו ערך dropout של 0.3 שזה הוא ערך מקובל אשר ימנע overfitting בצורה טובה וearly stopping של 10 שהוא גם ערך התחלתי מאוזן וטוב.

4. ניתוח של התוצאות מבחינת איכות הסיווג, זמן ריצה (והחומרה בה השתמשתם), מספר איטרציות, AUC, קצב התכנסות וכדומה. כולל גרפים לפי הצורך.

Classification Report (Test):				
	precision	recall	f1-score	support
Satisfied	0.98	0.92	0.95	8464
Dissatisfied	0.94	0.99	0.96	11018
accuracy			0.96	19482
macro avg	0.96	0.95	0.96	19482
weighted avg	0.96	0.96	0.96	19482



נתוני הסיווג היו טובים מאוד והתאימו למטרה המוצהרת בתרגיל. מטרתנו הייתה למצוא את הלקוחות שאינם מרוצים על מנת ללמוד מהמקרה ולנסות לשפר את השירות לפעם הבאה. בהתאם, קיבלנו תוצאות הכוללות ערך Recall ב Class 1 (אלה שאינם מרוצים) של 0.9875, מספר גבוה מאוד. Accuracy של 0.96, ערך המראה את המודל ככלל כחזק. וערך AUC של 0.9939. ערך גבוה מאוד המראה על מודל ברמה גבוהה. בסך הכל התוצאות מעידות על מודל החוזה בצורה טובה מאוד את ביקורות התעופה של הנוסעים.

הקוד הורץ בסביבת גוגל קולב כאשר המודלים "יער אקראי" ו"רגרסיה לוגיסטית" ב CPU בסרברים של גוגל ומודל הרשת הניורנית "Keras", הורץ על GPU T4 גם כן בסרברים של גוגל. הריצה של המודלים עצמם ערכה בין שניות לדקות בודדות (כאשר הכמות היותר גבוהה בעבור Keras).

בעת חיפוש ההיפר פרמטרים, הריצה ערכה זמנים מאוד גבוהים.

- חיפוש ההיפר פרמטרים של רגרסיה לוגית לקח כחצי שעה.
- חיפוש ההיפר פרמטרים של יער אקראי לקח כ4 שעות.
- חיפוש ההיפר פרמטרים של Keras לקח כ20 דקות.

בסך הכל זמני הריצה היו יחסית גבוהים וזאת בהתאם לכמות הגדולה של הדאטה והחיפוש הרחב אחרי היפר פרמטרים.

בריצה על רשת הניורונים, ההרצות נגמרו בערך לאחר רק 20 epochs בearly stopping והתכנסו בסבלנות 10 מוקדם מאוד, כאשר הROC המקסימלי היה ב10. זה מעיד על הגעה מהירה לתוצאה ויתר על כן על מניעת over fitting כאשר אם היינו נותנים לו להמשיך לרוץ למרות שכבר התכנס היינו מקבלים overfitting בוחק מתוך הריצה הממושכת על אותם ערכים. ברגרסה הלוגיסטית חלה התכנסות כבר ב29 למרות שהקצנו 500 איטרציות מקסימליות, כלומר הקוד רץ והתכנס מהר מאוד.

5. הסבירו כיצד חילקת את הדאטה לקבוצות training, validation, test. שימו לב שלא לערבב בין הקבוצות השונות.

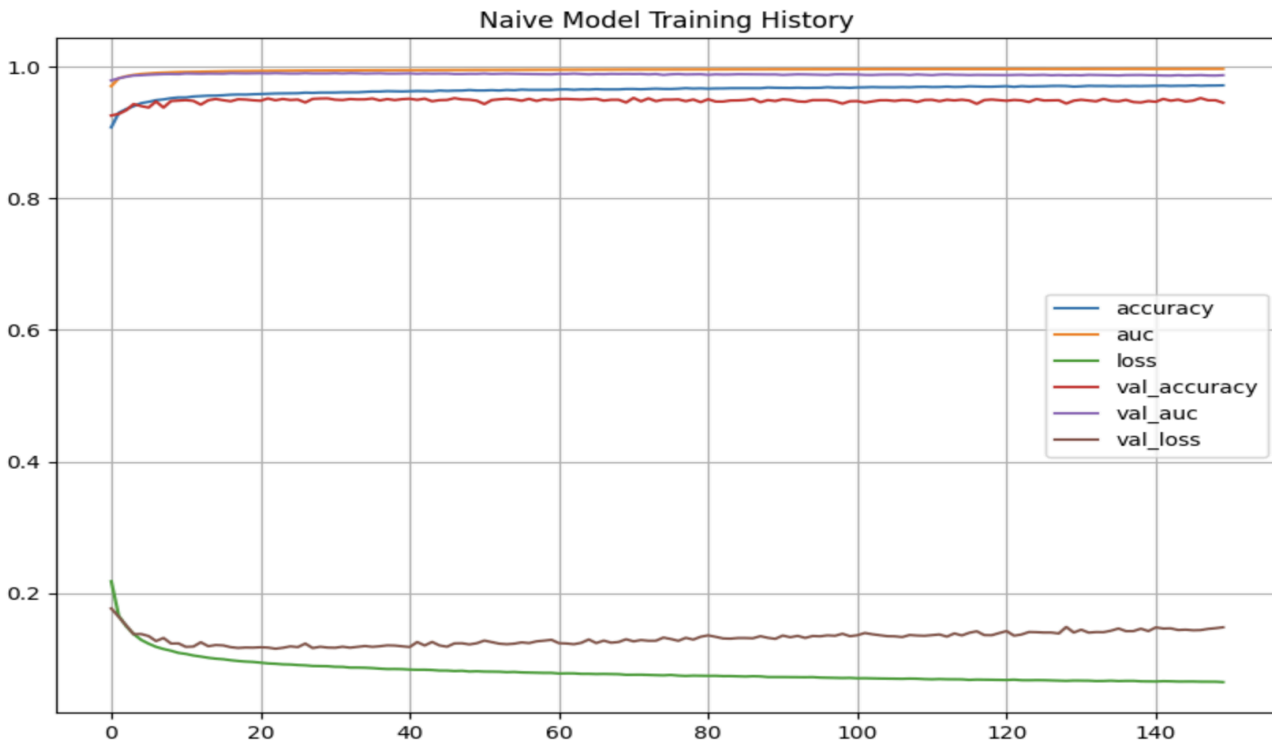
על מנת לבנות, לשפר ולבדוק באופן אמין חילקנו את הנתונים כמקובל ל3 קבוצות: training set, validation set, test set. הקפדנו על הפרדה מוחלטת בין הקבוצות על מנת למנוע data leakage שנוכל להעריך את המודל בצורה אמينة. הtraining set הוא הנתונים שהמודל לומד ממנה ומתאמן עליה, הvalidation set היא הקבוצה שמשמשת להערכה של המודל ושיפורו, ולבסוף הtest set באה לאחר סיום כל שלבי האימון אנו בודקים את התוצאות בהערכה סופית ואמינה. אנחנו חילקנו את הקבוצות כך ש70% מהנתונים יהיו לtraining, עוד 15% ילכו לvalidation וכך גם עוד 15% לtest. את החלוקה עשינו כך שבהתחלה חילקנו ל70% של training ו30% זמני ואז חילקנו את הזמני ל15% לכל אחת מ2 הקבוצות האחרות. בנוסף הקפדנו שהחלוקה תשמור על היחס המקורי של מרוצים ולא מרוצים על מנת לשמור על התפלגות זהה כך שכל קבוצה תהיה מייצגת.

6. הסבירו את בעיית over-fitting. הדגימו כיצד היא מתקבלת אצלכם בדאטה (אם מתקבלת). ממשו את הדרכים שלמדנו כדי למנוע זאת (dropout, early stopping, regularization). ציינו מה ההבדלים בין שיטות הרגולריזציה השונות, נסו שילובים שלהן ודווחו מה עבד הכי טוב.

Over fitting היא בעיה מרכזית ונפוצה שמתרחשת בלמידת מכונה. התופעה מתרחשת כאשר מודל למידה הוא מורכב וגמיש מדי ביחס לנתוני האימון, וכתוצאה מכך הוא אינו לומד את התבניות הכלליות והמהותיות בנתונים, אלא לומד את נתוני האימון יותר מדי, כולל את הרעש והסטייות המקריות הספציפיות למדגם האימון. לפי כך, מודל ששנמצא בover fitting יציג ביצועים כמעט מושלמים על נתוני האימון, אך יכולת ההכללה שלו לנתונים חדשים, שלא נצפו קודם לכן, תהיה ירודה. כתוצאה מכך, פונקציית הסיכון האמיתית של המודל, המודדת את ביצועיו על כלל האוכלוסייה, תהיה גבוהה יותר מזו של מודל פשוט יותר שהצליח ללמוד את המגמה הכללית מבלי להתייחס לרעשים.

כדי להדגים את תופעת over fitting באופן מעשי על הנתונים שלנו, בנינו מודל רשת נוירונים נאיבי. מודל זה נבנה ללא מנגנונים שמומנעים את זה כמו dropout או early stopping, ואומן עם מספר רב של איטרציות. המטרה הייתה לבחון כיצד המודל מתנהג ללא מנגנוני הגנה.

בגרף הבא מוצג תהליך הלמידה של המודל:



ניתן לראות בגרף סימנים לfitting קל מאוד וזאת על ידי הסתכלות על 2 הקווים בתחתית הגרף. מכיוון שבעוד שעקומת השגיאה על נתוני האימון (loss הקו הירוק) ממשיכה לרדת באופן עקבי, עקומת השגיאה על נתוני הולידציה (val loss הקו החום) יורדת בתחילה ואז מתייצבת ומתחילה לעלות קלות. הפער שנוצר בין שתי העקומות מוכיח שהמודל מתחיל לשנן את נתוני האימון ומאבד את יכולת ההכללה שלו לנתונים חדשים מפני שאמנם השגיאה יורדת לנתוני האימון אך לנתוני הולידציה השגיאה עולה. בנוסף חשוב לציין שהשגיאה של הולידציה עולה רק מעט מאוד, לכן אין כל כך הרבה overfitting במודל על הנתונים האלה.

לכן המדדים שמקבלים יוצאים גבוהים מאוד גם בלי שום טיפול בfitting over כפי שניתן לראות:

	precision	recall	f1-score	support
0	0.93	0.94	0.94	8464
1	0.96	0.95	0.95	11018
accuracy			0.95	19482
macro avg	0.94	0.95	0.94	19482
weighted avg	0.95	0.95	0.95	19482

ROC-AUC Score: 0.9902

כדי להתמודד עם בעיית over fitting שהודגמה, שילבנו במודל הסופי שלנו שתי טכניקות רגולריזציה מרכזיות:

1. Early Stopping: זוהי טכניקה שבה אנו עוקבים אחר ביצועי המודל על קבוצת הולידציה במהלך האימון. תהליך האימון נעצר באופן אוטומטי כאשר מדד הביצועים הנבחר val_auc מפסיק להשתפר במשך מספר קבוע של איטרציות. שיטה זו מונעת מהמודל להתאמן "יותר מדי" ולהיכנס למצב של $over\ fitting$, ומבטיחה שאנו שומרים את גרסת המודל שהשיגה את הביצועים הטובים ביותר על נתונים חדשים.
2. Dropout: טכניקה זו, שיושמה בין השכבות החבויות במודל שלנו, מכבה באופן אקראי אחוז מסוים של נוירונים בכל צעד אימון. פעולה זו מאלצת את הרשת לא להסתמך יתר על המידה על נוירונים ספציפיים, ומעודדת אותה ללמוד תבניות כלליות ועמידות יותר, מה שמשפר באופן משמעותי את יכולת ההכללה שלה.

לאחר שילוב מנגנוני הרגולריזציה עם $dropout=0.3$ ו $early\ stop=10$, אימנו את המודל מחדש. גרף האימון של המודל המשופר מוצג:



ניתן לראות כעת ששני העקומות בתחתית יורדות ביחד עד שהן מתיישרות בניגוד למקרה הקודם שהעקומה החומה התחילה לעלות. היעדר הפער ביניהן מעיד על כך שהמודל לומד את התבניות הכלליות בנתונים מבלי לשנן את הרעש, והטכניקות ששילבנו מנעו בהצלחה התאמת יתר.

ניתן לראות גם בתוצאות הסופיות שיפור קל על קבוצת הולידציה, בגלל שהבעיה פה לא כל כך גדולה השיפור גם הוא קטן:

Classification Report:				
	precision	recall	f1-score	support
0	0.96	0.93	0.95	8464
1	0.95	0.97	0.96	11018
accuracy			0.95	19482
macro avg	0.96	0.95	0.95	19482
weighted avg	0.95	0.95	0.95	19482

ROC-AUC Score: 0.9916

כלומר שיפור של 0.0014 בציון הAUC.

בנוסף השתמשנו בשיטת רוגלציה נוספת למניעת הבעיה בשיטת רגולציה L2. זו שיטה שמוסיפה "קנס" לפונקצית ההפסד של המודל על סמך סכום ריבועי המשקלים שלו. מטרת הקנס היא לעודד את המודל להשתמש במשקלים קטנים יותר, ובכך להעדיף פתרון פשוט יותר שנוטה פחות להתאים את עצמו לרעשים במדגם האימון אלא למשהו כללי יותר.

אלא התוצאות שקיבלנו בשיטה זו:

L2 Model Evaluation on Validation Set 609/609 1s 2ms/step

Classification Report:

	precision	recall	f1-score	support
0	0.95	0.90	0.92	8464
1	0.92	0.96	0.94	11018
accuracy			0.93	19482
macro avg	0.94	0.93	0.93	19482
weighted avg	0.93	0.93	0.93	19482

ROC-AUC Score: 0.9828

כפי שניתן לראות, המודל עם רגולריזציה L2 הציג ביצועים נמוכים יותר בהשוואה למודל שהשתמש בEarly Stopping וDropout ואפילו יחסית לזה שלא השתמש בכלל. ציון הAUC על קבוצת הולידציה עמד על 0.9835, נתון נמוך יותר מציון ה-AUC של 0.9916 שהושג על ידי מודל הקודם.

הסיבה הסבירה לכך היא שעוצמת הרגולריזציה שהפעלנו הייתה חזקה מדי עבור בעיה זו. הקנס הגבוה על המשקלים הגביל את גמישות המודל יתר על המידה, ומנע ממנו ללמוד את התבניות המורכבות בנתונים באותה רמת דיוק. מצב זה הוא *under fitting*.

מסקנת הניסוי היא שהשילוב של *Dropout* ו-*Early Stopping* היווה את האסטרטגיה היעילה והמאוזנת ביותר למניעת *over fitting* עבור מערך הנתונים שבחרנו.

7. מצאו את ההיפר-פרמטרים הכי טובים. הסבירו כיצד מצאתם.

עבור רגרסיה לוגית ההיפר פרמטרים האופטימליים עבורנו היו כך:

• **C** - 0.1767016940294795

• **L1 Ratio** - 1.0

• **Penalty** - elasticnet

הרצנו *StratifiedKFold* כאשר $k=5$ יחד עם חיפוש רנדומי שכלל 40 דגימות. השתמשנו ב-*Pipeline* שהריץ את ה-*Scaler*, אחריו *SMOTE*, ואז את הרגרסיה כאשר שמרנו בכל את המודל בעל ה-*AUC* הגבוה ביותר כאשר בסופו של דבר הגענו לערכים שלמעלה.

עבור יער אקראי ההיפר פרמטרים האופטימליים היו:

• **Max Depth** - 24

• **Max Features** - None

• **Minimum Leaf Samples** - 3

• **Minimum Samples Split** - 2

• **N Estimators** - 32

מצאנו את ההיפר-פרמטרים בדיוק באותה שיטה כמו רגרסיה לוגית. חלוקה ל-5 קיפולים, חיפוש רנדומי והרצת פייפליין.

נציין שהריצה של חיפוש זה ערכה שעות רבות והייתה הריצה הארוכה ביותר שהייתה לנו.

עבור רשת הניורונים *Keras* ההיפר פרמטרים האופטימליים היו:

• **Units1** - 224

• **Units2** - 128

• **Dropout** - 0.2

• **L2** - 0.0001

• **L3** - 0.005

מצאנו את ההיפר-פרמטרים באמצעות *KerasTuner* ובתוכו אופטימיזציה בייסיאנית העובדת בצורה הבאה. ראשית, בוחר *Tuner* קבוצת ערכים (לפי האפשרויות שהצענו לו) שמשמש בהם כהיפר פרמטרים ובונה מהם מודל, הוא מריץ אותו, שומר את ה-*AUC* (שוב אנחנו בוחרים את ההיפר פרמטרים לפי ה-*AUC* הטוב ביותר) ואת המודל עד למציאת מודל טוב יותר. לאורך החישוב, הוא יוצר מודל הסתברותי להערכת הפרמטרים הטובים ביותר להמשך.

המודל מעריך את תוצאת ההיפר פרמטרים הבאים לפי תהליך גאוסיאני, זהו מודל גאוסיאני שמגדיר התפלגות על פני פונקציות. המודל נוצר ובוחר כמות קטנה של נקודות על מנת להתחיל להעריך את המודל, הוא כעת מלמד את המודל (התהליך הגאוסיאני) ובהמשך מחפש את הערך המקסימלי לפי הפונקציה המוערכת בנקודה נתונה לוקחת $\mu(x)$ ממוצע משוער ואת רמת חוסר

הביטחון בערך בנקודה $\sigma(x)$. כעת הוא בוחר את הנקודה הבאה לפי הפונקציה (בנסיון לקבל ערך מקסימלי):

$$\alpha_{UCB}(x) = \mu(x) + \kappa\sigma(x)$$

המודל ממשיך וחוזר על חיפוש נקודות תוך שיפור עצמו עד ל-25 נסיונות (מספר אותו אנו הגדרנו) שבו כבר יש ערכי היפרפרמטריים טובים שכן האלגוריתם תמיד שב ומשפר את עצמו.